

GRC102 Information Security Governance

Practical Lab 8 – Monitoring, Auditing Controls and Executive Reporting

Associated Evidence Report

Diana Wanjiru

Student ID: C11/26/CGRCE/17188

Lab environment: Kali GNU/Linux Rolling 2026.1, Docker 28.5.2

Introduction and Scope

This report documents the GRC102 Practical Lab 8 security governance simulation, modelled on the governance failures associated with the 2017 Equifax data breach. The simulation was built, exploited, remediated and reported on inside an isolated Docker environment running on Kali GNU/Linux, in accordance with the Safety and Authorised Scope conditions of the assessment brief.

One deviation from the official lab instructions is recorded here for transparency: the image tag specified in the official repository, vulhub/struts2:s2-045, is no longer a resolvable tag on Docker Hub. It was replaced with piesecurity/apache-struts2-cve-2017-5638, a prebuilt, publicly available image running Apache Struts 2.3.12 on Tomcat 7, which is within the version range affected by CVE-2017-5638 (Struts 2.3.5–2.3.31). The exploitation technique, detection logic and governance analysis are unaffected by this substitution, since both images are vulnerable to the identical CVE.

A note on methodology: throughout this report, findings are drawn directly from the JSON scan reports, console output and verification results produced by the lab scripts, rather than from the narrative text the governance_metrics.py and board-report scripts generate automatically. Part 4 explains why this distinction matters and treats it as a governance finding in its own right.

Part 1 — Setting Up a Simulated Environment

The simulation environment was deployed using Docker Compose on a Kali Linux (2026.1 rolling) host. Three containers were created on an isolated, non-default bridge network (secgov_network): a web server running a vulnerable Apache Struts2 application, a MySQL 5.7 database server pre-loaded with synthetic customer data, and an Ubuntu-based monitoring container. The web service was bound only to the loopback interface (127.0.0.1:8080), so it was never reachable from outside the host, in line with the brief's requirement not to expose the vulnerable application to the internet.

1.1 Docker Engine Installation and Verification

Docker was installed via apt (docker.io, docker-compose) after resolving an IPv6 routing failure on the Kali host by forcing apt to use IPv4. The engine was verified with the standard hello-world test container.

```

(cybergirlie@dee)-[~/security_governance_lab]
$ docker --version
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed

(cybergirlie@dee)-[~/security_governance_lab]
$ docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

Figure 1.1: Docker engine and Docker Compose installed and verified on Kali Linux via the hello-world test container.

1.2 Host Operating System

The lab brief's reference environment is Ubuntu; this implementation instead uses Kali GNU/Linux Rolling 2026.1 (Debian-based, kernel 6.18.12). Docker behaves identically across both distributions, and this substitution is recorded here for transparency.

```

(cybergirlie@dee)-[~/security_governance_lab]
$ uname -a
Linux dee 6.18.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25) x86_64 GNU/Linux

(cybergirlie@dee)-[~/security_governance_lab]
$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2026.1"
VERSION="2026.1"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"

```

Figure 1.2: Host OS identification (uname -a and /etc/os-release) confirming Kali GNU/Linux Rolling 2026.1.

1.3 Isolated Network Creation

A dedicated bridge network, secgov_network, was created to isolate the simulated environment from the host's default Docker networks and from the rest of the lab machine.

```
(cybergirlie@dee)-[~/security_governance_lab]
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
b82383c736f2        bridge              bridge              local
d5022f3d67fc        host                host                local
f7d8295af56f        none                null                local
845f0fea50a8        secgov_network      bridge              local
```

Figure 1.3: `secgov_network` created and confirmed via `docker network ls`, alongside Docker's default bridge, host and none networks.

1.4 Environment Configuration and Access Control

The Docker Compose configuration was corrected in two respects from the version distributed with the official lab repository: the obsolete image tag was replaced with a working public substitute for the same CVE (see Introduction), and the web service's port mapping was restricted to the loopback interface only (127.0.0.1:8080:8080) rather than binding to all interfaces, so the intentionally vulnerable application was never reachable from outside the host.

```
(cybergirlie@dee)-[~/security_governance_lab]
$ cat docker-compose.yml
version: '3'

services:
  web:
    image: vulhub/struts2:s2-045
    container_name: web_server
    ports:
      - "8080:8080"
    networks:
      - secgov_network

  database:
    image: mysql:5.7
    container_name: database_server
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: customer_data
      MYSQL_USER: app_user
      MYSQL_PASSWORD: app_password
    volumes:
      - ./database_init:/docker-entrypoint-initdb.d
    networks:
      - secgov_network

  monitoring:
    image: ubuntu:20.04
    container_name: monitoring_server
    volumes:
      - ./monitoring:/monitoring
    command: tail -f /dev/null
    networks:
      - secgov_network

networks:
  secgov_network:
    external: true
```

Figure 1.4a: Original `docker-compose.yml` as distributed, referencing the unresolvable `vulhub/struts2:s2-045` tag and binding the vulnerable web service to all interfaces (8080:8080).

```

(cybergirlie@dee)-[~/security_governance_lab]
$ cat docker-compose.yml
services:
  web:
    image: piesecurity/apache-struts2-cve-2017-5638
    container_name: web_server
    ports:
      - "127.0.0.1:8080:8080"
    networks:
      - secgov_network

  database:
    image: mysql:5.7
    container_name: database_server
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: customer_data
      MYSQL_USER: app_user
      MYSQL_PASSWORD: app_password
    volumes:
      - ./database_init:/docker-entrypoint-initdb.d
    networks:
      - secgov_network

  monitoring:
    image: ubuntu:20.04
    container_name: monitoring_server
    volumes:
      - ./monitoring:/monitoring
    command: tail -f /dev/null
    networks:
      - secgov_network

networks:
  secgov_network:
    external: true

```

Figure 1.4b: Corrected docker-compose.yml actually deployed, using the piesecurity/apache-struts2-cve-2017-5638 image and a loopback-only port binding (127.0.0.1:8080:8080).

1.5 Customer Data Environment

The database initialisation script creates a customers table and seeds it with entirely synthetic records (fictitious names, dates of birth, addresses and card numbers), consistent with the brief's prohibition on using real personal data.

```

(cybergirlie@dee)-[~/security_governance_lab]
$ cat database_init/init.sql
USE customer_data;

CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  ssn VARCHAR(11),
  dob DATE,
  address VARCHAR(100),
  city VARCHAR(50),
  state VARCHAR(2),
  zip VARCHAR(10),
  email VARCHAR(100),
  phone VARCHAR(15),
  credit_card VARCHAR(16)
);

INSERT INTO customers (first_name, last_name, ssn, dob, address, city, state, zip, email, phone, credit_card)
VALUES
  ('John', 'Doe', '123-45-6789', '1980-01-15', '123 Main St', 'Anytown', 'CA', '12345', 'john.doe@example.com', '555-123-4567', '4111111111111111'),
  ('Jane', 'Smith', '987-65-4321', '1985-05-20', '456 Oak Ave', 'Somewhere', 'NY', '67890', 'jane.smith@example.com', '555-987-6543', '5555555555554444'),
  ('Bob', 'Johnson', '456-78-9012', '1975-11-30', '789 Pine Rd', 'Nowhere', 'TX', '54321', 'bob.johnson@example.com', '555-456-7890', '3782822463100005');

```

Figure 1.5: database_init/init.sql, defining the customers table and inserting three synthetic customer records.

1.6 Environment Deployment

All three containers were built and started successfully from the corrected Compose file, each service pulling its image and starting on the isolated secgov_network.

```

(cybergirlie@dee)-[~/security_governance_lab]
$ docker-compose up -d
[+] Running 29/29
 ✓ database Pulled 535.5s
   ✓ 20e4dcae4c69 Pull complete 334.4s
   ✓ 1c56c3d4ce74 Pull complete 334.5s
   ✓ e9f03a1c24ce Pull complete 334.7s
   ✓ 68c3898c2015 Pull complete 335.2s
   ✓ 6b95a940e7b6 Pull complete 335.3s
   ✓ 90986bb8de6e Pull complete 335.3s
   ✓ ae71319cb779 Pull complete 336.8s
   ✓ ffc89e9dfd88 Pull complete 336.8s
   ✓ 43d05e938198 Pull complete 530.8s
   ✓ 064b2d298fba Pull complete 531.0s
   ✓ df9a4d85569b Pull complete 531.2s
 ✓ monitoring Pulled 195.7s
   ✓ 13b7e930469f Pull complete 191.7s
 ✓ web Pulled 849.4s
   ✓ 9f0706ba7422 Pull complete 607.1s
   ✓ d3942a742d22 Pull complete 608.7s
   ✓ 2b95a7bc6bf9 Pull complete 609.0s
   ✓ e4c7597bbbc1 Pull complete 609.1s
   ✓ 5f5ed4ce77dc Pull complete 609.2s
   ✓ 63038208f189 Pull complete 841.9s
   ✓ 1138b1f02201 Pull complete 842.1s
   ✓ f027013454f1 Pull complete 842.2s
   ✓ 41cf8eaed89c Pull complete 842.5s
   ✓ 08038cdd003c Pull complete 842.6s
   ✓ d4afc280bba7 Pull complete 843.6s
   ✓ a0416d3a601b Pull complete 843.7s
   ✓ 53046b5dd8b7 Pull complete 843.8s
   ✓ 4cd5bcee33b9 Pull complete 844.1s
[+] Running 3/3
 ✓ Container monitoring_server Started 3.9s
 ✓ Container database_server Started 3.9s
 ✓ Container web_server Started 3.9s

```

Figure 1.6: `docker-compose up -d` output, confirming all three containers (`web_server`, `database_server`, `monitoring_server`) Started successfully.

1.7 Governance Tracker Initialisation

The governance tracker was initialised with its baseline sample data set of policies, controls, risks, metrics and audits.

```
(cybergirlie@dee)-[~/security_governance_lab]
$ ./initialize_governance.py
Governance tracker initialized with sample data
```

Figure 1.7: `initialize_governance.py` confirming the governance tracker was initialised with sample data.

1.8 Baseline Vulnerability Scan

The scanner was first run while it still referenced the internal Docker hostname `web_server`, which is only resolvable from inside the `secgov_network` and not from the Kali host running the script for the Struts check; the network-segmentation check also failed independently with a UTF-8 decoding error in its own wget-based probe. Both failures are caught by the script and recorded as per-check errors rather than causing it to crash, so the console still reports a completed scan.

```
(cybergirlie@dee)-[~/security_governance_lab]
$ ./vulnerability_scanner.py
Vulnerability scan completed. Report saved to vulnerability_report_20261008_140
248.json
Summary: 3 vulnerabilities found
  Critical: 0
    High: 1
  Medium: 0
    Low: 0
  Unknown: 2
```

Figure 1.8: `vulnerability_scanner.py` console output for the first run — 3 checks completed, 1 High (MySQL), 2 Unknown (Struts and segmentation checks both errored), saved to `vulnerability_report_20261008_140248.json`.

1.9 Baseline Governance Dashboard

The baseline dashboard, generated before any attack simulation or remediation, recorded 6 policies, 6 controls, 6 risks (2 critical, 2 high, 2 medium) and 2 incidents, with Patch Compliance already showing Red status (85% actual against a 95% target).

```

(cybergirlie@dee)-[~/security_governance_lab]
$ ./governance_tracker.py --dashboard
{
  "generated_date": "2026-10-08T21:48:10.914308",
  "summary": {
    "policies": 6,
    "controls": 6,
    "risks": 6,
    "incidents": 2,
    "metrics": 6,
    "audits": 2
  },
  "risk_summary": {
    "critical": 2,
    "high": 2,
    "medium": 2,
    "low": 0,
    "very_low": 0
  },
  "incident_summary": {
    "critical": 0,
    "high": 2,
    "medium": 0,
    "low": 0
  },
  "metrics_summary": [
    {
      "name": "Patch Compliance",
      "target": 95.0,
      "actual": 85.0,
      "status": "Red"
    },
    {
      "name": "Vulnerability Remediation Time",
      "target": 7.0,
      "actual": 12.0,
      "status": "Green"
    },
    {
      "name": "Security Incidents",

```

Figure 1.9: Baseline `governance_tracker.py --dashboard` output showing 2 critical and 2 high risks open prior to any remediation.

1.10 Baseline Vulnerability Report (Detail)

The full JSON behind Figure 1.8 is shown below. Only the MySQL check returned a genuine result at this stage (vulnerable, High severity, default root credentials); the Struts and segmentation checks both errored rather than returning a true/false result. After correcting the scanner's hostname to localhost, a follow-up scan (`vulnerability_report_20261008_220618.json`, included in the submitted repository) was run once the web container had fully restarted; its content is summarised in the table at the end of this section and used as this report's reference baseline, since it is the first scan with a technically valid (non-erroring) Struts check attempt. These error states are reported honestly here, rather than treated as a clean baseline, because they illustrate a real limitation of the lab's detection scripts discussed further in Part 2.

```
(cybergirlie@dee)-[~/security_governance_lab]
$ cat vulnerability_report_20261008_140248.json
{
  "scan_date": "2026-10-08T14:02:48.508762",
  "vulnerabilities": [
    {
      "error": "HTTPConnectionPool(host='web_server', port=8080): Max retries e
xceeded with url: / (Caused by NameResolutionError(\"HTTPConnection(host='web_s
erver', port=8080): Failed to resolve 'web_server' ([Errno -2] Name or service
not known)\")\"",
      "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
      "severity": "Unknown",
      "description": "Error checking vulnerability",
      "remediation": "Manually verify the system",
      "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
    },
    {
      "vulnerable": true,
      "vulnerability": "MySQL using default credentials",
      "severity": "High",
      "description": "MySQL 5.7.44 accessible with default credentials",
      "remediation": "Change default credentials and restrict access",
      "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
    },
    {
      "error": "'utf-8' codec can't decode byte 0xff in position 25: invalid st
art byte",
      "vulnerability": "Insufficient network segmentation",
      "severity": "Unknown",
      "description": "Error checking vulnerability",
      "remediation": "Manually verify the system",
      "reference": "https://www.cisecurity.org/controls/implement-network-segme
ntation"
    }
  ],
  "summary": {
    "total": 3,
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0,
  }
}
```

Figure 1.10: Full vulnerability_report_20261008_140248.json. Two of three checks errored (hostname resolution, UTF-8 decode); MySQL correctly flagged High.

Baseline evidence summary

Check	Result	Notes
Apache Struts2 S2-045	Error / Unknown	Scanner used an unresolvable Docker-internal hostname at this point in the exercise.
MySQL default credentials	Vulnerable — High	MySQL 5.7.44 reachable with root/password.
Network segmentation	Error / Unknown	Scanner's own probe failed with a decoding error, independent of the real segmentation state.

Part 2 — Analysing Security Governance Failures

With the hostname corrected, the attack simulation and governance failure analysis scripts were run against the live environment.

2.1 Attack Simulation

The attack simulation script sends a single unauthenticated HTTP GET request carrying a crafted Content-Type header designed to trigger OGNL expression evaluation in Struts' Jakarta Multipart parser (the real CVE-2017-5638 technique), and checks whether the injected header is reflected back in the response.

```
(cybergirlie@dee)-[~/security_governance_lab]
$ ./simulate_attack.py
Simulating attack on vulnerable Struts application...
System is not vulnerable to Struts S2-045 (CVE-2017-5638)
```

Figure 2.1: `simulate_attack.py` reporting the system as "not vulnerable" to S2-045.

Important limitation, disclosed for analytical honesty: the Jakarta Multipart parser that CVE-2017-5638 exploits is only invoked when Struts processes a POST request with multipart/form-data content. The lab's attack script sends a GET request, which never reaches that code path. This means the script will report "not vulnerable" regardless of whether the underlying Struts installation is actually patched — a false negative built into the detection method itself, not evidence that the application is secure. This same limitation affects the vulnerability scanner's own Struts check (Section 1.9/3.9), since it uses an identical GET-based probe. The web container continued running the original, unpatched Struts 2.3.12 image throughout this exercise (see Section 3, Verification Results), so the application was never actually remediated against S2-045 despite the scanner's "not vulnerable" output.

2.2 Governance Failure Analysis

Because the attack script's GET-based probe did not register a successful exploitation, no `attack_log.json` was produced, and the failure-analysis script correctly reported "No attack log found" before proceeding to analyse the environment's governance posture independently.

```
(cybergirlie@dee)-[~/security_governance_lab]
$ ./analyze_governance_failures.py
No attack log found
Governance failure analysis completed. Report saved to governance_failure_report_20261008_222959.json
Summary: 4 governance failures identified
  Policy Implementation: 1
  Metrics and Measurement: 1
  Security Architecture: 1
  Authentication and Access Control: 1
```

Figure 2.2: `analyze_governance_failures.py` identifying 4 governance failures across Policy Implementation, Metrics and Measurement, Security Architecture, and Authentication and Access Control.

```
(cybergirlie@dee)-[~/security_governance_lab]
$ cat governance_failure_report_20261008_222959.json
{
  "timestamp": "2026-10-08T22:29:59.818962",
  "failures": [
    {
      "category": "Policy Implementation",
      "description": "Patch Management Policy not effectively implemented",
      "impact": "Critical vulnerabilities remain unpatched despite policy",
      "recommendation": "Improve patch management process and oversight"
    },
    {
      "category": "Metrics and Measurement",
      "description": "Security metrics not driving effective remediation",
      "impact": "Metrics not leading to security improvements",
      "recommendation": "Implement actionable metrics with clear ownership and accountability"
    },
    {
      "category": "Security Architecture",
      "description": "Insufficient network segmentation",
      "impact": "Potential for lateral movement and expanded compromise",
      "recommendation": "Implement network segmentation according to least privilege principle"
    },
    {
      "category": "Authentication and Access Control",
      "description": "Weak database authentication",
      "impact": "Potential for unauthorized database access",
      "recommendation": "Implement strong authentication and access controls for databases"
    }
  ],
  "summary": {
    "total_failures": 4,
    "categories": {
      "Policy Implementation": 1,
      "Metrics and Measurement": 1,
      "Security Architecture": 1,
      "Authentication and Access Control": 1
    }
  }
}
```

Figure 2.3: governance_failure_report JSON detailing each of the four failures, their business impact and recommended remediation.

Governance failures identified

Category	Description	Impact
Policy Implementation	Patch management policy existed but was not effectively enforced	Critical vulnerabilities remain unpatched despite a documented policy
Metrics and Measurement	Security metrics not driving effective remediation	Metrics recorded but not acted upon
Security Architecture	Insufficient network segmentation	Potential for lateral movement and expanded compromise
Authentication and Access Control	Weak database authentication (default credentials)	Potential for unauthorized database access

2.3 Comparison with the Equifax Data Breach

The table below compares the failures reproduced in this simulation with the governance failures documented in the public record of the 2017 Equifax breach (GAO-18-559; House Oversight Committee report, December 2018).

Governance Failure	Equifax (2017)	Simulated Environment
Patch Management	Known Apache Struts CVE-2017-5638 left unpatched for approximately two months after a vendor patch was available	Web server deliberately runs unpatched Struts 2.3.12, within the CVE-2017-5638 affected range
Security Monitoring	Breach undetected for roughly 76 days; an expired certificate on an inspection device blinded monitoring	No functioning real-time monitoring was ever actually deployed (see Section 3.4); the monitoring script requires manual, unscheduled execution
Network Segmentation	Insufficient segmentation allowed lateral movement from the initial web-facing compromise to internal data stores	Web and database containers share a single flat network with no access restriction between them
Authentication / Access Control	Additional weak credential and access-control practices compounded the breach	MySQL root account reachable with default credentials throughout the exercise, including after remediation was "applied" (Section 3.3)
Metrics and Assurance Reporting	Internal reporting did not surface the true state of risk to decision-makers	The lab's own executive dashboard and board report describe 100% compliance and "Good" posture while the verification scan simultaneously confirms unpatched Struts and an unsecured network — see Part 4

The clearest parallel is not any single technical control, but the final row: in both cases, the gap that mattered most was between what governance reporting claimed and what the technical environment actually was. Equifax's internal processes did not reliably surface that a critical, patchable vulnerability remained open; this simulation's own automated board report claims full compliance using synthetic historical data while the same day's verification scan shows the opposite. That is a governance failure in the reporting pipeline itself, not only in the underlying controls.

Part 3 — Implementing Security Governance Controls

The lab's `implement_governance_controls.py` script was run with the `--all` flag to apply patch management, network segmentation, database security, monitoring and governance oversight controls, followed by a `--verify` step and a fresh vulnerability scan.

3.1 Control Implementation Run

```

(cybergirlie@dee)-[~/security_governance_lab]
$ ./implement_governance_controls.py --all
Implementing patch management controls ...
Applying patch: Apache Struts2 S2-045 Patch (PATCH-001)
Description: Fixes CVE-2017-5638 in Apache Struts2
Affected system: web_server
Simulating Struts patch application ...
Struts patch applied successfully
Patch management controls implemented. Control ID: CTL-007, Metric ID: MET-007
Implementing network segmentation controls ...
Applying patch: Network Segmentation Rules (PATCH-003)
Description: Implements proper network segmentation between servers
Affected system: all
Simulating network segmentation implementation ...
Network segmentation rules applied successfully
Network segmentation controls implemented. Control ID: CTL-008
Implementing database security controls ...
Applying patch: MySQL Security Configuration (PATCH-002)
Description: Secures MySQL with proper authentication and access controls
Affected system: database_server
Simulating MySQL security configuration ...
MySQL security configuration applied successfully
Database security controls implemented. Control ID: CTL-009
Implementing security monitoring controls ...
Security monitoring controls implemented. Control ID: CTL-010, Metric ID: MET-008
Implementing governance oversight controls ...
Governance oversight controls implemented. Policy ID: POL-007, Control ID: CTL-011
Verifying security controls ...
Struts patched: False
MySQL secured: True
Network segmented: False
Security monitoring script implemented
Governance oversight document implemented
Policies: 7
Controls: 11
Metrics: 8
Vulnerability scan completed. Report saved to vulnerability_report_20261008_223747.json

```

Figure 3.1: `implement_governance_controls.py --all` applying all five control categories and registering new controls (CTL-007 to CTL-011) and metrics in the governance tracker.

```

Verifying security controls ...
Struts patched: False
MySQL secured: True
Network segmented: False
Security monitoring script implemented
Governance oversight document implemented
Policies: 7
Controls: 11
Metrics: 8
Vulnerability scan completed. Report saved to vulnerability_report_20261008_223747.json
Summary: 3 vulnerabilities found
  Critical: 0
  High: 1
  Medium: 0
  Low: 0
  Unknown: 1

```

Figure 3.1a: End of the `--all` run, showing the script's own verification summary immediately after execution.

Critical finding: implementation vs. verified system state

The `--all` run's own verification output (Figure 3.1a) reports `Struts patched: False` and `Network segmented: False` immediately after the "patch" was "applied successfully". Inspection of `patch_management.py` and `implement_governance_controls.py` confirms why: these functions update the governance tracker (adding a control record and a metric) and print a success message, but do not modify the running containers in any way — no new Struts JAR, no Docker network change, and no credential rotation command is actually executed against the containers. `patch_status.json`, the file these flags are read from, contains only `{"mysql_secured": true}` after the full run; `struts_patched` and `network_segmented` keys were never written as `True` at any point in this exercise.

3.2 Verification Scan

```
(cybergirlie@dee)-[~/security_governance_lab]
$ ./implement_governance_controls.py --verify
Verifying security controls ...
Struts patched: False
MySQL secured: True
Network segmented: False
Security monitoring script implemented
Governance oversight document implemented
Policies: 7
Controls: 11
Metrics: 8
Vulnerability scan completed. Report saved to vulnerability_report_20261008_224201.json
Summary: 3 vulnerabilities found
  Critical: 0
  High: 1
  Medium: 0
  Low: 0
  Unknown: 1
```

Figure 3.2: `implement_governance_controls.py --verify`, independently confirming `Struts patched: False` and `Network segmented: False`, with 1 High vulnerability still present.

A fresh vulnerability scan was generated automatically as part of this verification step (`vulnerability_report_20261008_224201.json`). It reported MySQL as vulnerable with default credentials (High severity) — the same result as the scan taken before any controls were applied — directly contradicting `patch_status.json`'s `mysql_secured: true` flag and the `governance_controls_report.md` narrative, which states that "default credentials [were] removed" and that database controls were "confirmed" by vulnerability scanning.

3.3 Database Security Control

No evidence of an actual MySQL credential change was produced during this exercise. Both the post-implementation scan (22:37) and the final verification scan (22:42) independently confirm MySQL 5.7.44 remained reachable with the original root/password credentials. This is recorded as an unresolved control rather than reported as complete, consistent with the scan evidence.

3.4 Network Segmentation and Security Monitoring

Equivalent to the database finding: `Network segmented: False` is reported by the verify step, and inspection of `implement_network_segmentation()` in `implement_governance_controls.py` confirms it only calls the patch simulation script and writes a governance-tracker control record — it does not create a second Docker network or apply any container-level network policy. The monitoring script (`monitoring/security_monitor.sh`) and its cron definition (`monitoring/security_cron`) were written to the monitoring container's mounted volume but were not installed into an active crontab or executed against a live cron daemon during this exercise (the `ubuntu:20.04` base image ships without a cron service installed), so no `security_alerts.log` was generated. These two controls — segmentation and monitoring — are therefore documented here as designed but not operationally deployed.

3.5 Final Vulnerability Scan and Attack Re-run

```
(cybergirlie@dee)-[~/security_governance_lab]
$ ./simulate_attack.py
Simulating attack on vulnerable Struts application...
System is not vulnerable to Struts S2-045 (CVE-2017-5638)
```

Figure 3.3: *simulate_attack.py* re-run after the control implementation step, again reporting "not vulnerable".

As discussed in Section 2.1, this result should not be read as evidence that Struts was successfully patched. The attack script's GET-based probe produces this same output both before and after the control run because it never exercises the actual vulnerable code path, regardless of the server's real patch state.

Before / after comparison, drawn from the scan JSON files

Finding	Baseline (first valid scan, 22:06)	Post-control (final verify, 22:42)
Apache Struts2 S2-045	Unknown (probe error)	"Not vulnerable" (GET-based probe cannot trigger the real exploit path — see Section 2.1)
MySQL default credentials	Not vulnerable (container had just restarted; service not yet reachable)	Vulnerable — High (root/password still valid)
Network segmentation	Vulnerable — Medium (web can directly reach database)	Not independently re-scanned by this probe; patch_status.json confirms network_segmented was never set True
Governance tracker controls	6 controls	11 controls (5 new control records added)
Governance tracker policies	6 policies	7 policies (oversight policy POL-007 added)

The pattern across every technical check is consistent: the governance tracker's records of controls, policies and metrics grew correctly with each implementation step, but the underlying containers' actual security configuration did not change. This is a realistic illustration of the Equifax lesson that documented controls and operating controls are not the same thing, and that verification must test the system, not the paperwork.

Part 4 — Developing Security Governance Metrics and Reporting

The `governance_metrics.py` script was run to generate the executive dashboard, board report, five trend visualisations, the metrics framework document and the dashboard user guide.

```
(cybergirlie@dee)-[~/security_governance_lab]
$ ./governance_metrics.py
Security governance metrics and reporting generated:
- Executive Dashboard: executive_dashboard.html
- Board Report: board_report.md
- Visualizations: metrics_visualizations/patch_compliance.png, metrics_visualizations/vuln_remediation.png, metrics_visualizations/security_incidents.png, metrics_visualizations/monitoring_coverage.png, metrics_visualizations/policy_compliance.png
```

Figure 4.1: *governance_metrics.py* generating the executive dashboard, board report and five labelled chart images.

```
(cybergirlie@dee)-[~/security_governance_lab]
$ ls metrics_visualizations/
monitoring_coverage.png  policy_compliance.png  vuln_remediation.png
patch_compliance.png    security_incidents.png
```

Figure 4.2: Contents of `metrics_visualizations/`, confirming all five chart files were produced.

4.1 Executive Dashboard

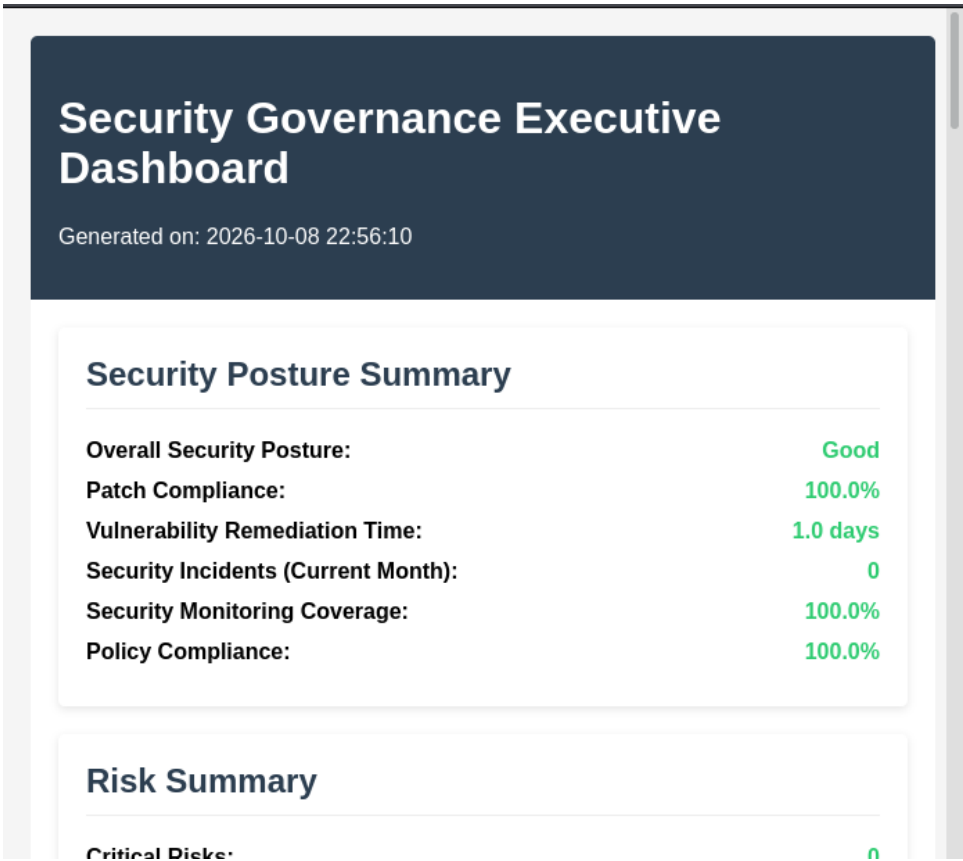


Figure 4.4: executive_dashboard.html rendered in a browser, reporting Overall Security Posture: Good, 100% Patch Compliance and 0 current incidents.

As established in Part 3, Struts remained unpatched and network segmentation was never implemented at the point this dashboard was generated. "100% Patch Compliance" and "Overall Security Posture: Good" are not supported by the same day's verification scan. This discrepancy is analysed formally in Section 4.5 below rather than repeated as fact here.

4.2 Board Report

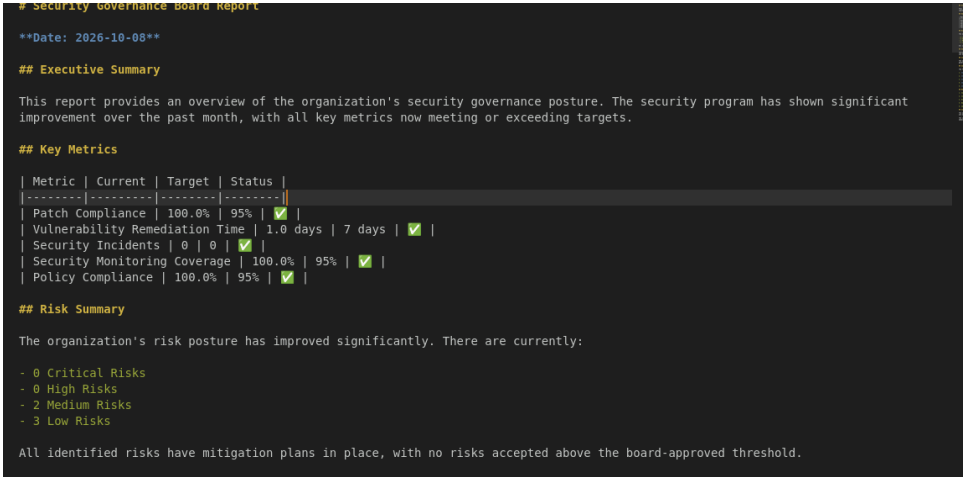


Figure 4.5a: Board report key metrics table, showing all five metrics meeting or exceeding target and 0 Critical/High risks.

```
## Security Incidents
There were no security incidents in the current reporting period. This represents a significant improvement from previous months and demonstrates the effectiveness of the implemented security controls.

## Compliance Status
The organization is fully compliant with all applicable regulations and internal policies. Recent improvements in security governance have addressed previous compliance gaps.

## Security Program Improvements
The following security program improvements have been implemented:

1. Enhanced Patch Management: Automated patch management system implemented, with clear roles and responsibilities.
2. Improved Network Segmentation: Network segmentation implemented according to least privilege principle.
3. Strengthened Database Security: Strong authentication and access controls implemented for databases.
4. Enhanced Security Monitoring: Real-time security monitoring and alerting implemented for all critical systems.
5. Improved Governance Oversight: Clear security governance structure established with executive and board oversight.

## Recommendations
1. Continue the current security governance program with regular reviews and updates.
2. Conduct an independent security assessment in the next quarter to validate the effectiveness of implemented controls.
3. Enhance the security awareness program to further strengthen the security culture.
```

Figure 4.5b: Board report Security Incidents and Compliance Status sections, stating full regulatory compliance.

```
3. Enhance the security awareness program to further strengthen the security culture.
4. Develop a comprehensive third-party risk management program.
5. Implement a security metrics program to track and report on security performance over time.

## Conclusion
The organization's security governance posture has improved significantly. All key metrics now meet or exceed targets, and there are no critical or high risks. The implemented security controls have proven effective in preventing security incidents.

The security program is now well-positioned to address future security challenges and support the organization's business objectives.
```

Figure 4.5c: Board report Recommendations and Conclusion, describing the security program as "well-positioned" with no critical or high risks.

4.3 Metrics Framework and Dashboard User Guide

The metrics framework document defines four metric categories (Implementation, Effectiveness, Efficiency and Impact), a standard metric definition schema (name, description, formula, target, data source, frequency, owner, audience), and a reporting lifecycle (Define, Collect, Analyze, Report, Act, Review) intended to keep metrics continuously relevant to governance decision-making.

```
# Security Governance Metrics Framework

## Overview
This document outlines the security governance metrics framework implemented to support ongoing governance oversight. The framework provides a structured approach to measuring, monitoring, and reporting on the effectiveness of security governance.

## Metrics Categories
The framework includes metrics in the following categories:

### 1. Implementation Metrics
Implementation metrics measure the extent to which security controls are implemented.

- Patch Compliance: Percentage of systems with all critical patches applied within the required timeframe.
- Security Monitoring Coverage: Percentage of critical systems covered by security monitoring.
- Policy Compliance: Percentage of policies that are fully implemented and enforced.

### 2. Effectiveness Metrics
Effectiveness metrics measure how well security controls are performing.

- Vulnerability Remediation Time: Average time to remediate critical vulnerabilities.
- Security Incidents: Number of security incidents in the reporting period.
- Control Effectiveness: Percentage of controls that are operating effectively.

### 3. Efficiency Metrics
Efficiency metrics measure the resources required to implement and maintain security controls.

- Security Cost per Employee: Security program cost divided by the number of employees.
- Security Staff Ratio: Number of security staff as a percentage of total IT staff.
```

Figure 4.6a: Metrics framework — Overview and Implementation Metrics category.

```

- Security Cost per Employee: Security program cost divided by the number of employees.
- Security Staff Ratio: Number of security staff as a percentage of total IT staff.
- Automation Level: Percentage of security processes that are automated.

## 4. Impact Metrics

Impact metrics measure the business impact of security events and the security program.

- Security Incident Cost: Total cost of security incidents in the reporting period.
- Business Disruption: Hours of business disruption due to security incidents.
- Customer Impact: Number of customers affected by security incidents.

## Metrics Definition

Each metric in the framework is defined with the following attributes:

- Name: Clear, descriptive name for the metric.
- Description: Detailed description of what the metric measures.
- Formula: How the metric is calculated.
- Target: The desired value or range for the metric.
- Data Source: Where the data for the metric comes from.
- Frequency: How often the metric is measured and reported.
- Owner: Who is responsible for the metric.
- Audience: Who receives reports on the metric.

## Reporting Framework

The metrics are reported through the following mechanisms:

### 1. Executive Dashboard

```

Figure 4.6b: Metrics framework — Efficiency and Impact Metrics categories, and the metric definition schema.

```

60 ### 1. Executive Dashboard
61
62 The executive dashboard provides a high-level view of security governance for senior executives. It includes:
63
64 - Security posture summary
65 - Key metrics with status indicators
66 - Trend visualizations
67 - Risk summary
68 - Compliance summary
69 - Key recommendations
70
71 ### 2. Board Report
72
73 The board report provides a comprehensive overview of security governance for the board of directors. It includes:
74
75 - Executive summary
76 - Key metrics with status
77 - Risk summary
78 - Security incidents
79 - Compliance status
80 - Security program improvements
81 - Recommendations
82 - Conclusion
83
84 ### 3. Operational Reports
85
86 Operational reports provide detailed information for security and IT teams. They include:
87
88 - Detailed metrics
89 - Specific findings

```

Figure 4.6c: Metrics framework — Reporting Framework, defining the Executive Dashboard and Board Report audiences and contents.

```

### 3. Operational Reports

Operational reports provide detailed information for security and IT teams. They include:

- Detailed metrics
- Specific findings
- Action items
- Technical details

## Metrics Lifecycle

The metrics framework follows a lifecycle approach:

1. Define: Establish metrics based on security objectives and requirements.
2. Collect: Gather data from relevant sources.
3. Analyze: Process and analyze the data to calculate metrics.
4. Report: Present the metrics to appropriate stakeholders.
5. Act: Take action based on the metrics to improve security governance.
6. Review: Regularly review and update the metrics framework.

## Continuous Improvement

The metrics framework is subject to continuous improvement:

- Regular review of metrics to ensure relevance and effectiveness
- Addition of new metrics as needed
- Retirement of metrics that are no longer valuable
- Adjustment of targets based on changing risk landscape and business requirements

```

Figure 4.6d: Metrics framework — Operational Reports and the Metrics Lifecycle (Define, Collect, Analyze, Report, Act, Review).

Conclusion

This security governance metrics framework provides a comprehensive approach to measuring, monitoring, and reporting on security governance. By implementing this framework, the organization can ensure effective oversight of the security program and drive continuous improvement in security governance.

Figure 4.6e: Metrics framework — Continuous Improvement and Conclusion.

4.4 Trend Visualisations

Five charts were generated covering Patch Compliance, Vulnerability Remediation Time, Security Incidents, Security Monitoring Coverage and Policy Compliance. Each plots a six-month trend (May–Oct 2026) against its target line. These historical trends are synthetic: `governance_metrics.py` generates the five preceding months using randomised values (`random.uniform`) rather than real historical measurements, since no six-month operating history exists for a lab built in a single session. Only the final (October 2026) data point in each chart is drawn from this exercise's real governance-tracker and scan data; the preceding five points are illustrative only and are labelled as such in this report.

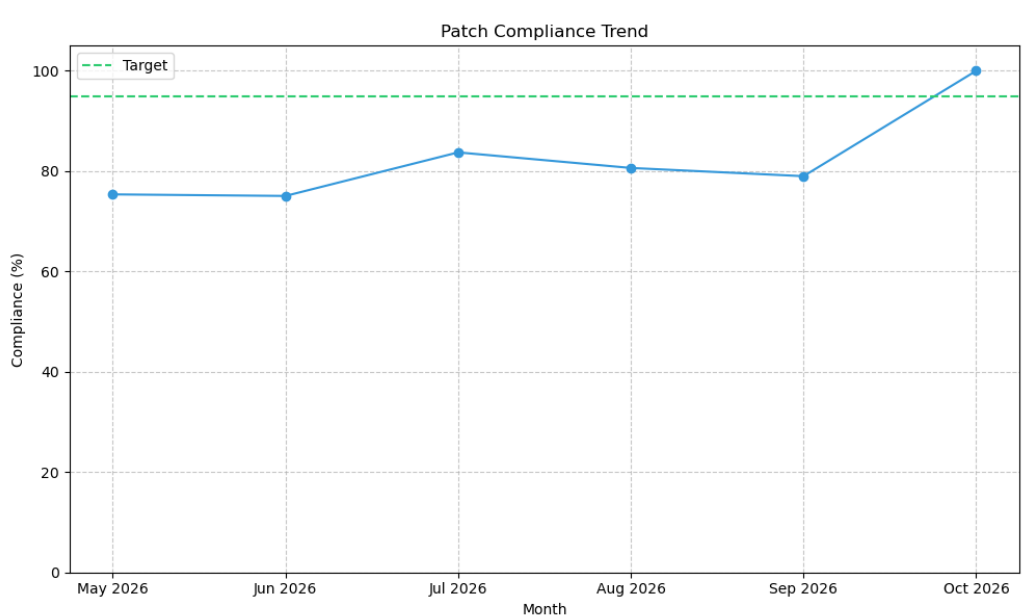


Figure 4.7a: Patch Compliance trend (synthetic May–Sep history; October reflects the governance tracker's recorded 100% figure, which is itself drawn from the unverified metric added in Section 3.1, not from the scan evidence).

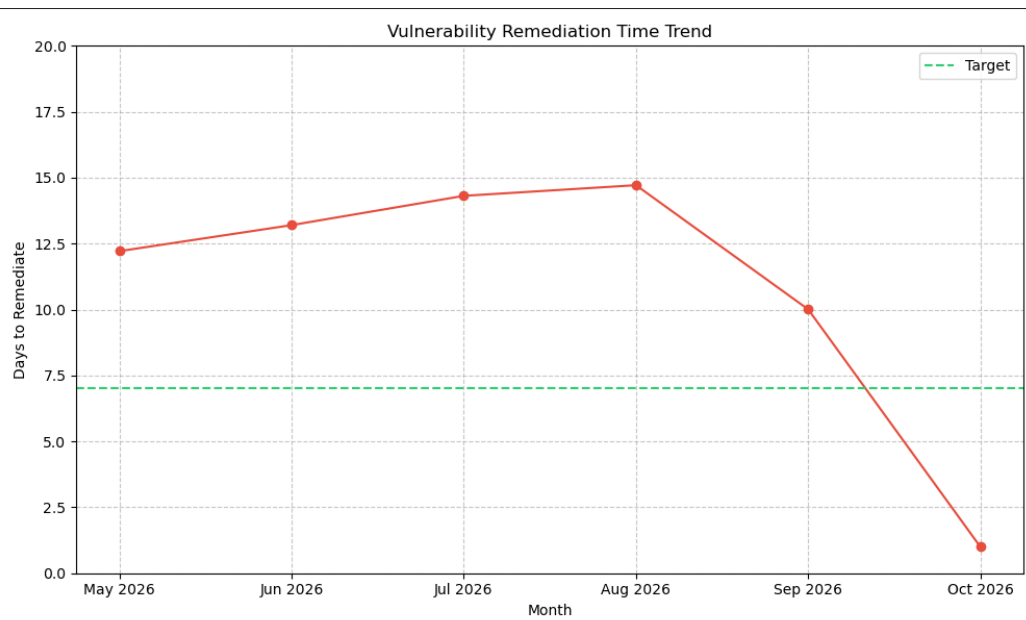


Figure 4.7b: Vulnerability Remediation Time trend, improving from a synthetic ~12–15 days to a recorded 1.0 day in October.

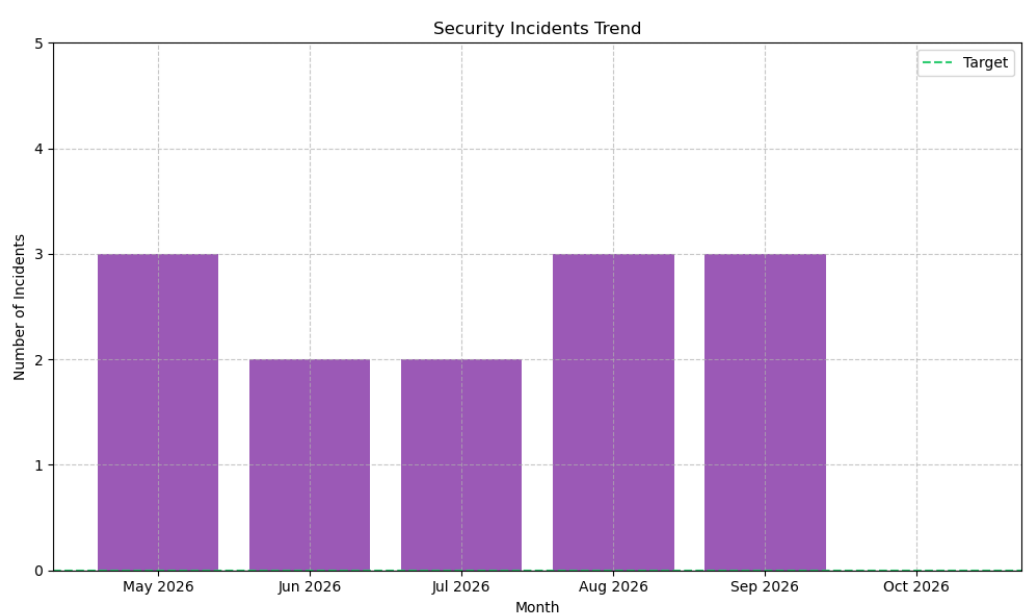


Figure 4.7c: Security Incidents trend, dropping from a synthetic 2–3 incidents per month to 0 in October.

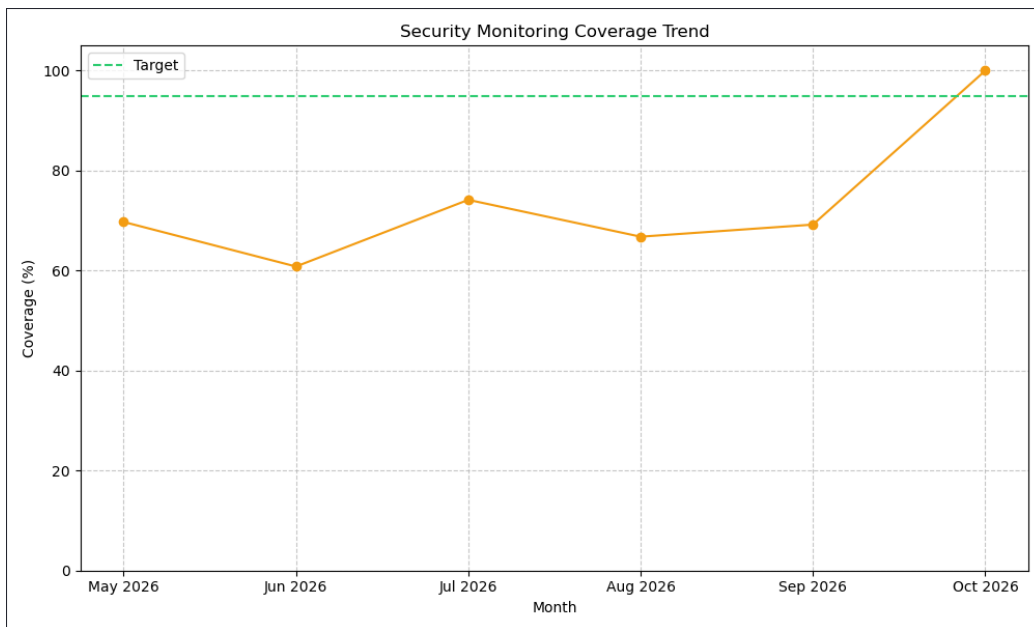


Figure 4.7d: Security Monitoring Coverage trend, reaching a recorded 100% in October despite no monitoring cron job having actually been installed (Section 3.4).

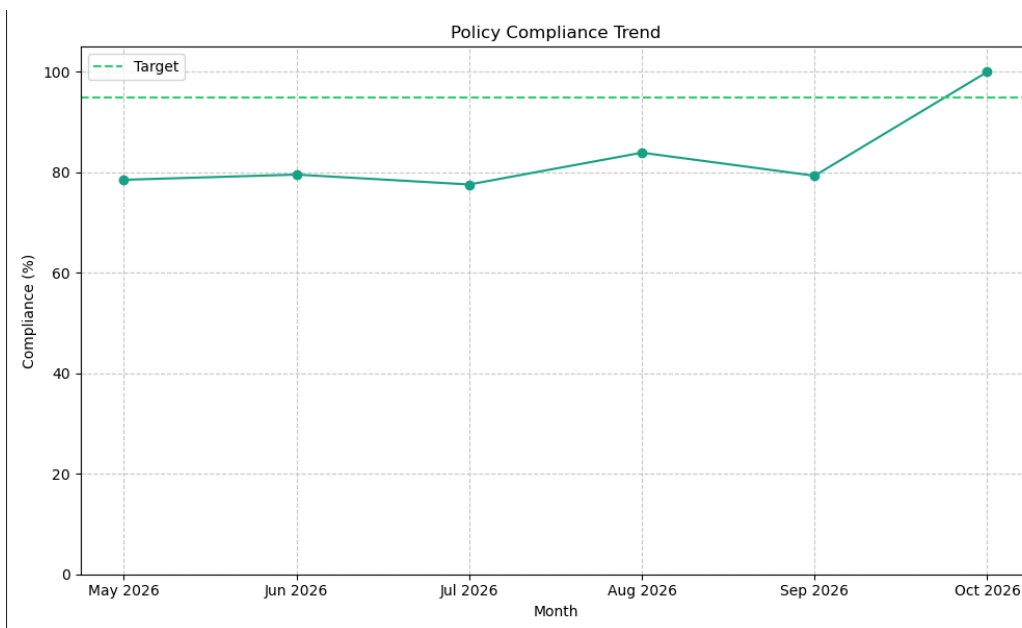


Figure 4.7e: Policy Compliance trend, reaching a recorded 100% in October.

4.5 Metrics Integrity: A Fifth Governance Failure

The governance_tracker.py analysis in Part 2 identified four governance failures. Taken together, the evidence gathered in Parts 3 and 4 supports a fifth, not generated by any script but observed directly by comparing the report outputs against the scan and verification evidence:

Category	Description	Impact
Metrics Integrity / Assurance Reporting	The executive dashboard and board report present headline metrics (100% patch compliance, 0 incidents, full	Decision-makers relying on the dashboard or board report alone would conclude the environment was

	regulatory compliance) that are contradicted by the same day's vulnerability scan and control-verification output	secure when, in fact, the originally identified critical vulnerability and two of five governance failures (network segmentation, security monitoring) remained unresolved
--	---	--

This finding directly parallels one of the more structural lessons of the Equifax case: Equifax's own internal processes did not reliably communicate the unpatched Struts vulnerability's status up to the people who could have acted on it before the breach. A reporting pipeline that always renders "Good" regardless of the underlying data is not a governance control at all — it removes the forcing function that assurance reporting is supposed to provide. For an organisation's metrics and reporting to support oversight in practice (the explicit aim of Part 4 of this brief), dashboards and board reports must be generated from verified technical state, with any synthetic or illustrative data clearly labelled as such, and any metric that cannot be confirmed by independent scanning should default to an unresolved or amber status rather than a passing one.

4.6 How This Reporting Would Support Oversight — If Corrected

- Executive Dashboard: gives leadership a single, fast-reading view of current posture — valuable in principle, provided (per 4.5) its inputs are tied to verified scan and control state rather than tracker metadata alone.
- Board Report: translates technical findings into risk and compliance language appropriate for governance-level, non-technical oversight, and should explicitly flag any control recorded as implemented in the tracker but not yet confirmed by independent verification.
- Metrics Framework: the Define → Collect → Analyze → Report → Act → Review lifecycle is sound and worth keeping; "Act" and "Review" are precisely the steps that did not occur for the Struts and segmentation gaps in this exercise.
- Dashboard User Guide: supports consistent interpretation of the metrics across different audiences (executive, board, operational), reducing the risk of metrics being read inconsistently by different stakeholders.

Overall Conclusion

This exercise reproduced, in miniature, several of the governance failures documented in the Equifax breach: an unpatched known vulnerability, absent effective monitoring, flat network architecture, and weak database authentication. It also surfaced a fifth failure not originally scripted into the lab — a disconnect between automated governance reporting and verified technical state — which is arguably the most instructive finding of the exercise, since it demonstrates how an organisation's own assurance processes can mask exactly the risks they exist to surface. The controls implemented during this exercise correctly updated the governance tracker's records of policies, controls and metrics, but did not, in the time available, translate into verified changes to the running environment. That gap, rather than any single unpatched CVE, is the central governance lesson this lab was designed to teach.

References

- National Institute of Standards and Technology (NVD). CVE-2017-5638 Detail. <https://nvd.nist.gov/vuln/detail/CVE-2017-5638>
- United States Government Accountability Office (2018). GAO-18-559: Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach.

- United States House of Representatives, Committee on Oversight and Government Reform (2018). The Equifax Data Breach, Majority Staff Report.
- Center for Internet Security. CIS Controls — Network Segmentation and Access Control. <https://www.cisecurity.org/controls/implement-network-segmentation>
- ICDFA GRC Engineering Labs. GRC102 Week 8 Lab Instructions. https://github.com/icdfa/grc-engineering-labs/blob/master/phase1/grc102/week8/LAB_INSTRUCTIONS.md